Zaawansowane techniki programowania

Laboratorium 6

ZMIENDAK WIKTOR

ZADANIA OBOWIĄZKOWE:

1. Implementujemy zapytanie GET renderujące widok books/list oraz pokazujące listę książek:

```
public BooksController(LibraryService libraryService) {
    this.libraryService = libraryService;
}

@GetMapping("/books")
public String books(Model model) {
    model.addAttribute("books", libraryService.getBooks());
    return "books/list";
}

@GetMapping("/books/new")
public String newBookForm(Model model) {
    if (!model.containsAttribute("bookForm")) {
        model.addAttribute("bookForm", new BookForm());
    }
    return "books/form";
}
```

```
public BookListItem toBookListItem(Book book) {
    return new BookListItem(
            book.getId(),
            book.getTitle(),
            book.getAuthor(),
            book.getTotalCopies(),
            book.getAvailableCopies(),
            book.getAvailableCopies() > 0);
}
```

2. Implementujemy zapytanie POST tworzące książki i redirectujące do /books.

```
@PostMapping("/books")
public String createBook(@Valid @ModelAttribute("bookForm") BookForm bookForm,
        BindingResult bindingResult) {
    if (bindingResult.hasErrors()) {
        return "books/form";
    }
    libraryService.createBook(bookForm);
    return "redirect:/books";
}
```

3. Implementujemy bean validation dla formularza dodawania książek.

```
@NotBlank(message = "Title is required")
private String title;

@NotBlank(message = "Author is required")
private String author;

@Min(value = 1, message = "Total copies must be at least 1")
private int totalCopies;
```

4. Implementujemy zapytanie GET renderujące formularz z wybraną książką.

```
@GetMapping("/loans")
public String loans(@RequestParam(name = "active", defaultValue = "false") boolean active,
        Model model) {
    // TODO (4.0): dodaj do modelu listę wypożyczeń i flagę activeOnly
    return "loans/list";
}

@GetMapping("/loans/new")
public String newLoanForm(@RequestParam("bookId") long bookId, Model model) {
    model.addAttribute("selectedBook", libraryService.getBook(bookId));
    if (!model.containsAttribute("loanForm")) {
        model.addAttribute("loanForm", new LoanForm(bookId, borrower: ""));
    }
    return "loans/form";
}
```

5. Implementujemy zapytanie POST tworzące wyporzyczenie i zmniejszające ilość dostępnych egzemplarzy.

```
@PostMapping("/loans")
public String createLoan(@Valid @ModelAttribute("loanForm") LoanForm loanForm,
        BindingResult bindingResult,
        Model model) {
    if (bindingResult.hasErrors()) {
        if (loanForm.getBookId() != null) {
            model.addAttribute(attributeName: "selectedBook", libraryService.getBook(loanForm.getBookId()));
        }
        return "loans/form";
    }

    try {
        libraryService.createLoan(loanForm);
        return "redirect:/loans";
    } catch (IllegalStateException ex) {
        bindingResult.reject(errorCode: "loan.unavailable", ex.getMessage());
        model.addAttribute(attributeName: "selectedBook", libraryService.getBook(loanForm.getBookId()));
        return "loans/form";
    }
}
```

6. Tworzymy walidację dostępnych egzemplarzy.

```
@Override
public void createLoan(LoanForm form) {
    Book book = getBook(form.getBookId());
    if (book.getAvailableCopies() <= 0) {
        throw new IllegalStateException(s: "No copies available");
    }
```

ZADANIA DODATKOWE:

1. Implementujemy zapytanie GET dla dasboardu który pokazuje różne parametry na zasadzie agregatu.

```
@GetMapping({ "/", "/dashboard" })
public String dashboard(Model model) {
    model.addAttribute(attributeName: "dashboard", libraryService.getDashboard());
    return "dashboard";
}
```

2. Implementujemy dla zapytania GET pokazywanie tylko aktywnych wypożyczeni.

```
@GetMapping("/loans")
public String loans(@RequestParam(name = "active", defaultValue = "false") boolean active,
        Model model) {
    model.addAttribute(attributeName: "loans", libraryService.getLoans(active));
    model.addAttribute(attributeName: "activeOnly", active);
    return "loans/list";
}
```

3. Implementujemy zapytanie POST loans z id które zwiększa liczbę dostępnych egzemplarzy.

```java
@PostMapping("/loans/{loanId}/return")
public String returnLoan(@PathVariable long loanId) {
    libraryService.returnLoan(loanId);
    return "redirect:/loans";
}
```

4.

```java
@Override
public void createLoan(LoanForm form) {
    Book book = getBook(form.getBookId());
    if (book.getAvailableCopies() <= 0) {
        throw new IllegalStateException(s: "No copies available");
    }

    LocalDate borrowedOn = LocalDate.now(clock);
    Loan loan = new Loan();
    loan.setId(store.nextLoanId());
    loan.setBookId(book.getId());
    loan.setBorrower(form.getBorrower());
    loan.setBorrowedOn(borrowedOn);
    loan.setDueDate(borrowedOn.plusDays(daysToAdd: 14));
    loan.setReturned(returned: false);

    book.setAvailableCopies(book.getAvailableCopies() - 1);
    store.saveLoan(loan);
    store.saveBook(book);
}
```